

VOLUME 3

Certificats · Ingress · Azure APIM

Gestion TLS/SSL, cert-manager, Ingress NGINX, Azure Application Gateway et API Management



Certificats TLS/SSL



Ingress Controller



App Gateway AGIC



Azure APIM
Intégration



Sécurité Réseau



ArgoCD +
cert-manager

① Gestion des Certificats TLS/SSL — cert-manager & Azure Key Vault

cert-manager automatise tout le cycle de vie des certificats TLS dans Kubernetes : émission, renouvellement et révocation. Cette section couvre l'installation via Helm, la configuration des ClusterIssuers Let's Encrypt (HTTP-01 pour clusters publics, DNS-01 via Azure DNS pour clusters privés ou wildcards), l'import de certificats d'entreprise depuis Azure Key Vault via le CSI Driver, et les commandes de diagnostic pour déboguer une émission bloquée.

Composant / Étape	Commande / YAML	Description
Installer cert-manager	<pre># Via Helm helm repo add jetstack https://charts.jetstack.io helm repo update helm install cert-manager jetstack/cert-manager \ --namespace cert-manager \ --create-namespace \ --version v1.14.0 \ --set installCRDs=true \ --set global.leaderElection.namespace=cert-manager # Vérifier l'installation kubectl get pods -n cert-manager kubectl get crds Select-String 'cert-manager' # CRDs attendus: certificates, clusterissuers, issuers, # certificaterequests, challenges, orders</pre>	cert-manager = controller K8s pour automatiser TLS Gère : émission, renouvellement, révocation Supporté : Let's Encrypt, Azure Key Vault, self-signed, CA privée InstallCRDs=true : installe les définitions de ressources
ClusterIssuer — Let's Encrypt (HTTP-01 challenge)	<pre>apiVersion: cert-manager.io/v1 kind: ClusterIssuer metadata: name: letsencrypt-prod spec: acme: server: https://acme-v02.api.letsencrypt.org/directory email: admin@mon-domaine.com privateKeySecretRef: name: letsencrypt-prod-key solvers: - http01: ingress: class: nginx --- # Staging (pour tests - pas de rate limit) apiVersion: cert-manager.io/v1 kind: ClusterIssuer metadata: name: letsencrypt-staging spec: acme: server: https://acme-staging-v02.api.letsencrypt.org/directory email: admin@mon-domaine.com privateKeySecretRef: name: letsencrypt-staging-key solvers: - http01: ingress: class: nginx</pre>	HTTP-01 : Let's Encrypt vérifie via /.well-known/acme-challenge/ Nécessite que le port 80 soit accessible depuis Internet Toujours tester avec staging avant prod Rate limit prod : 5 certificats/domaine/semaine
ClusterIssuer — DNS-01 challenge (Azure DNS)	<pre># Créer Service Principal pour Azure DNS \$sp = az ad sp create-for-rbac \ --name sp-cert-manager \ --role 'DNS Zone Contributor' \ --scopes \$(az network dns zone show -g rg-prod -n mon-domaine.com --query id -o tsv) \ ConvertFrom-Json kubectl create secret generic azure-dns-config \ --from-literal=client-secret=\$sp.password \ -n cert-manager # ClusterIssuer DNS-01 apiVersion: cert-manager.io/v1 kind: ClusterIssuer metadata: name: letsencrypt-dns spec: acme: server: https://acme-v02.api.letsencrypt.org/directory email: admin@mon-domaine.com privateKeySecretRef: name: letsencrypt-dns-key solvers: - dns01: azureDNS: subscriptionID: '<sub-id>' resourceGroupName: rg-prod hostedZoneName: mon-domaine.com environment: AzurePublicCloud clientID: \$sp.appId clientSecretSecretRef: name: azure-dns-config key: client-secret tenantID: '<tenant-id>'</pre>	DNS-01 : pas besoin du port 80 ouvert Idéal pour clusters internes / privés Supporte les wildcard : *.mon-domaine.com Azure DNS = zone DNS managée dans Azure

Composant / Étape	Commande / YAML	Description
Émettre un certificat manuellement	<pre>apiVersion: cert-manager.io/v1 kind: Certificate metadata: name: mon-app-tls namespace: mon-app spec: secretName: mon-app-tls-secret duration: 2160h # 90 jours renewBefore: 360h # Renouveler 15 jours avant subject: organizations: [Mon Organisation] commonName: mon-app.mon-domaine.com dnsNames: - mon-app.mon-domaine.com - api.mon-domaine.com issuerRef: name: letsencrypt-prod kind: ClusterIssuer # Vérifier le statut kubectl get certificate -n mon-app kubectl describe certificate mon-app-tls -n mon-app kubectl get certificaterequest -n mon-app</pre>	secretName : Secret K8s créé automatiquement Avec clés tls.crt et tls.key reNewBefore : renouvellement automatique 15j avant dnsNames : SANs (Subject Alternative Names) Statut READY=True = certificat valide
Certificat depuis Azure Key Vault (AKV + CSI Driver)	<pre># Importer un certificat dans Key Vault az keyvault certificate import \ --vault-name kv-mon-app \ --name mon-app-cert \ --file ./mon-app.pfx \ --password \$certPassword # Ou créer un certificat dans AKV (auto-signé ou CA) az keyvault certificate create \ --vault-name kv-mon-app \ --name mon-app-cert \ --policy '\$(az keyvault certificate get-default-policy)' # SecretProviderClass pour monter le certificat apiVersion: secrets-store.csi.x-k8s.io/v1 kind: SecretProviderClass metadata: name: azure-kv-certs namespace: mon-app spec: provider: azure parameters: keyvaultName: kv-mon-app objects: array: - objectName: mon-app-cert objectType: secret # pour récupérer clé+cert objectVersion: '' secretObjects: - secretName: tls-from-akv type: kubernetes.io/tls data: - objectName: mon-app-cert key: tls.crt - objectName: mon-app-cert key: tls.key</pre>	Centraliser les certificats dans Azure Key Vault Renouvellement automatique avec DigiCert/Entrust CSI Driver monte le certificat comme Secret K8s TLS Idéal pour certificats d'entreprise (CA interne, EV...)
Inspecter & Debugger	<pre># État des certificats kubectl get certificates -A kubectl get challenges -A kubectl get orders -A # Détails du certificat kubectl describe cert mon-app-tls -n mon-app # Contenu du secret TLS kubectl get secret mon-app-tls-secret -n mon-app -o jsonpath='{.data.tls\.crt}' base64 -d openssl x509 -text -noout # Logs cert-manager kubectl logs -n cert-manager deploy/cert-manager -f # Forcer renouvellement kubectl delete certificaterequest -n mon-app --all cmctl renew mon-app-tls -n mon-app</pre>	cmctl = CLI officielle cert-manager openssl x509 -text : voir dates d'expiration, SANs Les challenges = preuve que vous contrôlez le domaine Si challenge bloqué : vérifier DNS + firewall port 80

② Ingress Controller — NGINX & Azure Application Gateway (AGIC)

L'Ingress Controller est la passerelle HTTP/HTTPS qui route le trafic entrant vers les bons services Kubernetes. Cette section compare NGINX (léger, flexible, open source) et AGIC/Azure Application Gateway (WAF intégré, autoscaling Azure natif), avec des exemples concrets : Ingress basique, HTTPS automatique via cert-manager, routing multi-services par host et path, rate limiting et protection WAF avec ModSecurity/OWASP.

Composant	Commande / YAML	Description & Cas d'usage
Installer Ingress NGINX (sur AKS)	helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx helm repo update helm install ingress-nginx ingress-nginx/ingress-nginx \ --namespace ingress-nginx \ --create-namespace \ --set controller.replicaCount=2 \ --set controller.nodeSelector.'kubernetes.io/os'=linux \ --set controller.service.annotations.'service.beta.kubernetes.io/azure-load-balancer-health-probe-request-path'=/healthz # Récupérer l'IP publique kubectl get svc -n ingress-nginx ingress-nginx-controller # EXTERNAL-IP → configurer dans Azure DNS	NGINX = Ingress Controller le plus répandu Crée automatiquement un Azure Load Balancer (Standard) replicaCount=2 → haute disponibilité IP externe → configurer record A dans Azure DNS
Ingress basique (HTTP)	apiVersion: networking.k8s.io/v1 kind: Ingress metadata: name: mon-app-ingress namespace: mon-app annotations: nginx.ingress.kubernetes.io/rewrite-target: / spec: ingressClassName: nginx rules: - host: mon-app.mon-domaine.com http: paths: - path: / pathType: Prefix backend: service: name: mon-app-svc port: number: 80	ingressClassName: nginx → sélectionne le controller NGINX host : doit correspondre au DNS configuré rewrite-target : réécrire le chemin si nécessaire PathType Prefix : /api/ match /api/users /api/posts...
Ingress HTTPS + cert-manager	apiVersion: networking.k8s.io/v1 kind: Ingress metadata: name: mon-app-ingress-tls namespace: mon-app annotations: nginx.ingress.kubernetes.io/ssl-redirect: 'true' nginx.ingress.kubernetes.io/force-ssl-redirect: 'true' cert-manager.io/cluster-issuer: 'letsencrypt-prod' nginx.ingress.kubernetes.io/proxy-body-size: '50m' nginx.ingress.kubernetes.io/proxy-read-timeout: '120' spec: ingressClassName: nginx tls: - hosts: - mon-app.mon-domaine.com secretName: mon-app-tls-secret rules: - host: mon-app.mon-domaine.com http: paths: - path: / pathType: Prefix backend: service: name: mon-app-svc port: number: 80	cert-manager.io/cluster-issuer : annotation magique → cert-manager génère automatiquement le certificat ssl-redirect : force HTTPS (301) proxy-body-size : taille max upload (50MB) proxy-read-timeout : timeout pour requêtes longues

Composant	Commande / YAML	Description & Cas d'usage
Ingress multi-services / routing avancé	apiVersion: networking.k8s.io/v1 kind: Ingress metadata: name: multi-app-ingress namespace: mon-app annotations: cert-manager.io/cluster-issuer: letsencrypt-prod nginx.ingress.kubernetes.io/use-regex: 'true' nginx.ingress.kubernetes.io/configuration-snippet: more_set_headers 'X-Frame-Options: SAMEORIGIN'; more_set_headers 'X-Content-Type-Options: nosniff'; spec: ingressClassName: nginx tls: - hosts: [mon-domaine.com, api.mon-domaine.com] secretName: multi-tls-secret rules: - host: mon-domaine.com http: paths: - path: / pathType: Prefix backend: service: { name: frontend-svc, port: { number: 80 } } - path: /api/v1 pathType: Prefix backend: service: { name: backend-svc, port: { number: 8080 } } - host: api.mon-domaine.com http: paths: - path: / pathType: Prefix backend: service: { name: api-svc, port: { number: 3000 } }	Un seul Ingress peut router vers plusieurs services Basé sur Host ET/OU Path Headers sécurité : X-Frame-Options, X-Content-Type-Options use-regex : paths avec expressions régulières Eviter de trop charger un seul Ingress en prod
Azure Application Gateway Ingress Controller (AGIC)	# Activer AGIC sur AKS az aks enable-addons \ --resource-group rg-prod \ --name aks-prod \ --addons ingress-appgw \ --appgw-name appgw-prod \ --appgw-subnet-cidr '10.2.0.0/24' # Ingress AGIC apiVersion: networking.k8s.io/v1 kind: Ingress metadata: name: mon-app-agic namespace: mon-app annotations: kubernetes.io/ingress.class: azure/application-gateway appgw.ingress.kubernetes.io/ssl-redirect: 'true' appgw.ingress.kubernetes.io/backend-protocol: 'http' appgw.ingress.kubernetes.io/waf-policy-for-path: '/subscriptions/.../applicationGatewayWebApplicationFirewallPolicies/waf-policy' spec: tls: - hosts: [mon-app.mon-domaine.com] secretName: mon-app-tls rules: - host: mon-app.mon-domaine.com http: paths: - path: / pathType: Prefix backend: service: name: mon-app-svc port: number: 80	AGIC = Azure App Gateway managé par K8s Avantages vs NGINX : • WAF (Web Application Firewall) intégré • Terminaison TLS sur App Gateway • Autoscaling natif Azure • Intégration Azure Monitor native Inconvénient : plus lent à provisionner
Rate Limiting & WAF NGINX	# ConfigMap global NGINX apiVersion: v1 kind: ConfigMap metadata: name: ingress-nginx-controller namespace: ingress-nginx data: limit-req-status-code: '429' use-gzip: 'true' gzip-types: 'application/json text/plain text/css application/javascript' keep-alive: '75' keep-alive-requests: '1000' # Annotations sur l'Ingress pour rate limiting annotations: nginx.ingress.kubernetes.io/limit-rpm: '100' nginx.ingress.kubernetes.io/limit-burst-multiplier: '5' nginx.ingress.kubernetes.io/limit-connections: '10' nginx.ingress.kubernetes.io/enable-modsecurity: 'true' nginx.ingress.kubernetes.io/enable-owasp-core-rules: 'true'	limit-rpm : 100 requêtes/minute par IP limit-burst-multiplier : pic autorisé (×5 = 500 burst) ModSecurity + OWASP Core Rules = WAF open source gzip : compression pour réduire la bande passante Appliqué par annotation sur un Ingress spécifique

③ Azure API Management (APIM) — Intégration AKS & OpenShift

Azure APIM est la couche de gouvernance des APIs exposées par vos microservices Kubernetes : authentification OAuth2/JWT (Azure AD), rate limiting, quotas, caching, transformation des requêtes/réponses et monitoring centralisé. Cette section couvre la création via PowerShell, le déploiement d'un Self-Hosted Gateway directement dans AKS pour les APIs internes, l'import de specs OpenAPI, la configuration des politiques XML, la gestion des souscriptions et clés API, et l'intégration Application Insights.

Opération	Commande PowerShell / Bicep / Portail	Description
Créer APIM	<pre># PowerShell Az module New-AzApiManagement \ -ResourceGroupName rg-prod \ -Location eastus \ -Name apim-mon-organisation \ -Organization 'Mon Organisation' \ -AdminEmail admin@mon-domaine.com \ -Sku Developer # ou Standard, Premium # Vérifier Get-AzApiManagement -ResourceGroupName rg-prod # Azure CLI az apim create \ --resource-group rg-prod \ --name apim-mon-organisation \ --publisher-email admin@mon-domaine.com \ --publisher-name 'Mon Organisation' \ --sku-name Developer</pre>	SKUs : • Developer : test/dev (pas de SLA) • Basic : petites organisations • Standard : production, 1 unit • Premium : multi-région, VNet injection Déploiement : 30-45 minutes
Intégrer AKS → APIM (Self-Hosted Gateway)	<pre># Créer un Self-Hosted Gateway dans APIM \$ctx = New-AzApiManagementContext \ -ResourceGroupName rg-prod \ -ServiceName apim-mon-organisation New-AzApiManagementGateway \ -Context \$ctx \ -GatewayId 'aks-gateway' \ -Description 'Gateway déployé sur AKS' # Récupérer la config K8s du gateway \$gwKey = Get-AzApiManagementGatewayKey \ -Context \$ctx -GatewayId 'aks-gateway' # Créer le Secret K8s pour la clé gateway kubectl create secret generic apim-gateway-token \ --from-literal=value="GatewayKey \$(\$gwKey.PrimaryKey)" \ -n apim # Déployer le Self-Hosted Gateway sur AKS kubectl apply -f https://raw.githubusercontent.com/Azure/api-management-self-hosted-gateway/main/deploy/kubernetes.yml -n apim</pre>	Self-Hosted Gateway = pod APIM dans votre cluster Idéal pour APIs internes non exposées sur Internet Communication sécurisée entre APIM cloud et gateway local Même politiques APIM appliquées dans le cluster
Importer une API depuis AKS	<pre># Importer OpenAPI/Swagger depuis l'app \$ctx = New-AzApiManagementContext -ResourceGroupName rg-prod -ServiceName apim-mon-org # Depuis une URL OpenAPI Import-AzApiManagementApi \ -Context \$ctx \ -SpecificationFormat OpenApi \ -SpecificationUrl 'https://mon-app.mon-domaine.com/swagger.json' \ -Path 'mon-api' \ -ApiId 'mon-api-v1' # Depuis un fichier local Import-AzApiManagementApi \ -Context \$ctx \ -SpecificationFormat OpenApi \ -SpecificationPath './openapi.yaml' \ -Path 'mon-api' \ -ApiId 'mon-api-v1' # Vérifier Get-AzApiManagementApi -Context \$ctx</pre>	Importer depuis Swagger/OpenAPI 3.0 APIM crée automatiquement les opérations Path = préfixe URL dans APIM Après import : configurer backend URL vers le service K8s

Opération	Commande PowerShell / Bicep / Portail	Description
Configurer Backend (AKS Service)	<pre># Configurer le backend APIM → service K8s \$ctx = New-AzApiManagementContext -ResourceGroupName rg-prod -ServiceName apim-mon-org # URL backend = URL interne AKS via VNet ou Ingress New-AzApiManagementBackend \ -Context \$ctx \ -BackendId 'aks-mon-app-backend' \ -Url 'http://mon-app-svc.mon-app.svc.cluster.local/' -Protocol Http # Ou via l'Ingress HTTPS public New-AzApiManagementBackend \ -Context \$ctx \ -BackendId 'aks-ingress-backend' \ -Url 'https://mon-app.mon-domaine.com/' \ -Protocol Https # Lier l'API au backend via policy Set-AzApiManagementPolicy \ -Context \$ctx \ -ApiId 'mon-api-v1' \ -Policy '<policies><inbound><base /><set-backend-service base-url="http://mon-app-svc.mon-app.svc.cluster.local" /></inbound></policies>'</pre>	Backend = URL cible derrière APIM Option 1 : URL interne (APIM dans le même VNet via VNet injection) Option 2 : Ingress public HTTPS Recommandé : APIM Premium + VNet injection + URL interne
Politiques APIM essentielles	<pre><!-- Politique complète : authentification + rate limit + transform --> <policies> <inbound> <base /> <!-- 1. Valider JWT (AAD) --> <validate-jwt header-name='Authorization' failed-validation-httpcode='401'> <openid-config url='https://log in.microsoftonline.com/<tenant-id>/well-known/openid-configurati on' /> <required-claims> <claim name='aud' match='any'> <value>api://<app-client-id></value> </claim> </required-claims> </validate-jwt> <!-- 2. Rate limiting par souscription --> <rate-limit calls='100' renewal-period='60' /> <!-- 3. Quota journalier --> <quota calls='10000' renewal-period='86400' /> <!-- 4. Ajouter header backend --> <set-header name='X-API-Gateway' exists-action='override'> <value>APIM-Prod</value> </set-header> <!-- 5. Cache --> <cache-lookup vary-by-developer='false' vary-by-developer-groups='false'> <vary-by-header>Accept</vary-by-header> </cache-lookup> </inbound> <backend><base /></backend> <outbound> <base /> <!-- 6. Masquer les headers internes --> <set-header name='X-Powered-By' exists-action='delete' /> <cache-store duration='3600' /> </outbound> <on-error><base /></on-error> </policies></pre>	Les politiques APIM = middleware de l'API Gateway validate-jwt : authentification via Azure AD rate-limit : 100 appels/minute par souscription quota : 10000 appels/jour cache-lookup/store : réduire la charge sur le backend Ordre d'exécution : inbound → backend → outbound → on-error

Opération	Commande PowerShell / Bicep / Portail	Description
Souscriptions & Clés API	# Créer un produit APIM New-AzApiManagementProduct \ -Context \$ctx \ -ProductId 'api-standard' \ -Title 'API Standard' \ -Description 'Accès aux APIs de production' \ -State Published \ -SubscriptionRequired \$true # Ajouter l'API au produit Add-AzApiManagementApiToProduct \ -Context \$ctx \ -ProductId 'api-standard' \ -ApiId 'mon-api-v1' # Créer une souscription New-AzApiManagementSubscription \ -Context \$ctx \ -ProductId 'api-standard' \ -Name 'Client-App-A' \ -UserId 'admin' \ -State Active # Récupérer les clés Get-AzApiManagementSubscriptionKey \ -Context \$ctx \ -SubscriptionId '<sub-id>'	Produit = regroupement d'APIs avec conditions d'accès Souscription = clé API pour un client/application Clé envoyée dans header : Ocp-Apim-Subscription-Key Limites par produit : différents tiers (Free/Standard/Premium)
APIM + Azure AD (OAuth 2.0)	# 1. Enregistrer l'app backend dans AAD \$app = az ad app create \ --display-name 'aks-mon-app-api' \ --identifiant-uris 'api://mon-app-api' ConvertFrom-Json # 2. Configurer APIM pour valider les tokens AAD \$oauth = New-AzApiManagementAuthorizationServer \ -Context \$ctx \ -ServerId 'aad-oauth2' \ -Name 'Azure AD OAuth2' \ -ClientId \$app.appId \ -AuthorizationEndpoint \ "https://login.microsoftonline.com/<tenant>/oauth2/v2.0/authorize" \ -TokenEndpoint \ "https://login.microsoftonline.com/<tenant>/oauth2/v2.0/token" \ -ClientAuthenticationMethod @('Body') \ -GrantTypes @('authorizationCode','clientCredentials') # 3. Policy validate-jwt dans l'API # (voir politique ci-dessus : validate-jwt)	OAuth 2.0 + PKCE pour les clients SPA/Mobile Client Credentials pour les services M2M APIM valide le JWT avant de transmettre au backend Le backend reçoit des requêtes déjà authentifiées
Monitoring APIM	# Activer Application Insights \$ai = New-AzApplicationInsights \ -ResourceGroupName rg-prod \ -Name ai-apim -Location eastus New-AzApiManagementLogger \ -Context \$ctx \ -LoggerId 'ai-logger' \ -InstrumentationKey \$ai.InstrumentationKey \ -Name 'App Insights Logger' # Requetes metriques APIM (PowerShell) \$metrics = Get-AzMetric \ -ResourceId \$(Get-AzApiManagement -ResourceGroupName rg-prod -Name apim-mon-org).Id \ -MetricName 'TotalRequests','SuccessfulRequests','FailedRequests','Capacity' \ -TimeGrain 00:05:00 \ -StartTime (Get-Date).AddHours(-1) \ -EndTime (Get-Date) \$metrics ForEach-Object { Write-Host "\$(\$_.Name.Value): \$(\$_.Data[-1].Total)" }	Application Insights = APM complet pour APIM Métriques : requêtes totales, erreurs, latence, capacité Logs : détails requête/réponse, headers, body Tableau de bord Azure Monitor intégré Alertes sur taux d'erreur ou capacité > 80%

④ Architecture de Référence Complète — Flux Requête vers Application

Vue d'ensemble des 8 couches traversées par une requête HTTP depuis Internet jusqu'aux pods applicatifs : DNS Azure → DDoS Protection → APIM → Ingress Controller → Service K8s → Pods avec HPA → Secrets via Key Vault/ESO → Base de données via Private Endpoint. Chaque couche est décrite avec son rôle, le composant Azure/K8s correspondant et la commande de vérification.

Couche	Composant Azure/K8s	Rôle dans le flux	Configuration clé
1 — DNS	Azure DNS Zone publique	Résolution mon-app.mon-domaine.com → IP publique du Load Balancer Record A ou CNAME vers l'IP de l'Ingress	<pre>az network dns record-set a add-record \ -g rg-prod -z mon-domaine.com \ -n mon-app \ -a <INGRESS-EXTERNAL-IP></pre>
2 — WAF / DDoS	Azure DDoS Protection Azure Front Door (optionnel) App Gateway WAF	Protection DDoS niveau réseau Front Door : CDN + WAF global + routing WAF : protection OWASP Top 10	<pre>az network ddos-protection create \ -g rg-prod -n ddos-plan \ -l eastus # Associer au VNet az network vnet update \ -g rg-prod -n aks-vnet \ --ddos-protection true \ --ddos-protection-plan ddos-plan</pre>
3 — Azure APIM	API Management (Premium + VNet)	Gateway API centralisé Auth OAuth2/JWT, rate limiting, caching, transformation, monitoring	<pre># VNet injection Premium az apim update \ --resource-group rg-prod \ --name apim-mon-org \ --virtual-network Internal # Internal = APIM privé dans le VNet</pre>
4 — Ingress Controller	NGINX Ingress ou AGIC (App Gateway)	Routing HTTP/HTTPS vers les services K8s Terminaison TLS (cert-manager) Load balancing, rate limit	<pre>kubectl get ingress -n mon-app kubectl describe ingress mon-app-ingress -n mon-app # → Rules, TLS, Backend endpoints</pre>
5 — Service K8s	Service ClusterIP Service ExternalName	Load balancing interne entre les pods DNS : mon-app-svc.mon-app.svc.cluster.local	<pre>kubectl get endpoints mon-app-svc -n mon-app # Voir les IPs des pods cibles</pre>
6 — Pods (App)	Deployment (3+ replicas) HPA auto-scaling	Exécution de l'application HPA scale selon CPU/mémoire Rolling update zéro downtime	<pre>kubectl top pods -n mon-app kubectl get hpa -n mon-app # REPLICAS, CPU%, MEMORY%</pre>
7 — Secrets & Config	Azure Key Vault External Secrets Operator ConfigMap	Secrets injectés via ESO depuis AKV Configuration non-sensible via ConfigMap Workload Identity = pas de mots de passe statiques	<pre>kubectl get externalsecret -n mon-app kubectl get secret app-secrets -n mon-app # Les pods voient des env vars normales</pre>
8 — Base de Données	Azure Database PostgreSQL Azure SQL / Cosmos DB Private Endpoint	BDD accessible uniquement via VNet Service ExternalName K8s = alias DNS Ne jamais exposer la BDD sur Internet	<pre>kubectl get svc postgres-svc -n mon-app # TYPE: ExternalName # EXTERNAL-IP: mon-postgres.postgres.database.azure.com</pre>

⑤ Checklist Sécurité & Bonnes Pratiques Réseau

Cinq domaines de durcissement à appliquer systématiquement en production : NetworkPolicy (isolation inter-pods et inter-namespaces), Pod Security Standards (non-root, capabilities dropped), RBAC (un ServiceAccount par app, audit des droits larges), scan CVE des images avec Trivy ou Microsoft Defender, et activation des logs d'audit Kubernetes pour détecter les accès suspects aux secrets.

Domaine	Pratique	Commande / Vérification
Network Policy	Isoler les namespaces et contrôler le trafic inter-pods. Principe du moindre privilège réseau.	<pre># Bloquer tout le trafic entrant par défaut apiVersion: networking.k8s.io/v1 kind: NetworkPolicy metadata: name: default-deny-ingress namespace: mon-app spec: podSelector: {} policyTypes: [Ingress] --- # Autoriser uniquement depuis l'Ingress apiVersion: networking.k8s.io/v1 kind: NetworkPolicy metadata: name: allow-from-ingress namespace: mon-app spec: podSelector: matchLabels: app: mon-app ingress: - from: - namespaceSelector: matchLabels: kubernetes.io/metadata.name: ingress-nginx</pre>
Pod Security Standards	Appliquer les standards de sécurité K8s. Empêcher l'escalade de privilèges.	<pre># Appliquer Pod Security Standard au namespace kubectl label namespace mon-app \ pod-security.kubernetes.io/enforce=restricted \ pod-security.kubernetes.io/warn=restricted # securityContext dans chaque pod securityContext: runAsNonRoot: true runAsUser: 1000 fsGroup: 2000 seccompProfile: type: RuntimeDefault capabilities: drop: [ALL]</pre>
RBAC	Accès minimum requis. Un ServiceAccount par application. Auditer régulièrement.	<pre># Vérifier les permissions kubectl auth can-i --list \ --as=system:serviceaccount:mon-app:mon-app-sa \ -n mon-app # Auditer les ClusterRoleBindings larges kubectl get clusterrolebindings -o json \ jq '.items[] select(.roleRef.name=="cluster-admin") .subjects'</pre>
Scan vulnérabilités images	Scanner toutes les images avant déploiement. Bloquer si CVE critique.	<pre># Avec Trivy (open source) trivy image acrprod.azurecr.io/mon-app:v1.2.0 \ --exit-code 1 \ --severity CRITICAL,HIGH # Activer Microsoft Defender for Containers az aks update -g rg-prod -n aks-prod \ --enable-defender</pre>
Audit Logs	Activer les logs d'audit K8s. Alerter sur les accès suspects.	<pre># Logs d'audit AKS az aks update -g rg-prod -n aks-prod \ --enable-azure-monitor-metrics # Requête : accès secrets KubeAuditEvent where ObjectRef_resource == 'secrets' and Verb in ('get','list','watch') summarize count() by User_username, TimeGenerated</pre>

■ Architecture recommandée pour la production : Internet → Azure DDoS → Azure Front Door → APIM (Premium + VNet) → Ingress NGINX (cert-manager TLS) → Services K8s → Pods (Workload Identity) → Azure Database via Private Endpoint. Tous les secrets passent par Azure Key Vault via External Secrets Operator.

⑥ ArgoCD & cert-manager — Gestion GitOps des Certificats TLS

ArgoCD ne génère pas de certificats TLS — c'est le rôle de cert-manager. En revanche, ArgoCD orchestre tout le cycle de vie des objets cert-manager (ClusterIssuer, Certificate, SecretProviderClass) depuis Git : il les déploie, les protège contre toute suppression manuelle grâce au self-heal, et garantit l'ordre d'installation via les Sync Waves. Cette section explique comment les deux outils se complètent et fournit la structure de repo Git de référence.

Rôles Respectifs : qui fait quoi ?

Outil	Ce qu'il fait	Ce qu'il ne fait PAS
cert-manager	• Émet les certificats TLS (ACME Let's Encrypt, Azure Key Vault, CA interne) • Renouvelle automatiquement avant expiration • Stocke le certificat dans un Secret K8s (tls.crt / tls.key) • Répond aux challenges HTTP-01 et DNS-01 • Surveille en continu la validité des certificats	• Ne gère pas les déploiements d'applications • Ne synchronise pas depuis Git • Ne protège pas contre la suppression accidentelle de ses propres objets
ArgoCD	• Déploie et synchronise les objets cert-manager depuis Git (ClusterIssuer, Certificate, Ingress annotés) • Protège via self-heal : si un ClusterIssuer est supprimé manuellement, ArgoCD le recrée • Installe cert-manager lui-même (chart Helm) via une Application dédiée • Ordonne l'installation via Sync Waves (cert-manager avant les apps qui en dépendent) • Historise tous les changements de configuration TLS dans Git	• Ne génère pas de certificats • Ne parle pas à Let's Encrypt / Azure Key Vault • Ne gère pas le cryptographique
Combinés ensemble	• ArgoCD déploie cert-manager + les ClusterIssuers depuis Git • ArgoCD déploie les Ingress avec annotation cert-manager.io/cluster-issuer • cert-manager détecte l'annotation et émet le certificat • ArgoCD self-heal garantit que les objets cert-manager ne dérivent pas • Tout l'état TLS est versionné dans Git → disaster recovery simplifié	• Ne remplacent pas un HSM ou une PKI d'entreprise pour les environnements très réglementés

Structure du Repo Git k8s-config (référence)

Dossier / Fichier	Contenu	Géré par ArgoCD ?
bootstrap/root-app.yaml	# Application racine – appliquer une seule fois apiVersion: argoproj.io/v1alpha1 kind: Application metadata: name: root-app namespace: argocd spec: source: repoURL: https://github.com/org/k8s-config path: bootstrap/apps targetRevision: main destination: server: https://kubernetes.default.svc namespace: argocd syncPolicy: automated: selfHeal: true prune: true	Non — appliqué manuellement une seule fois pour démarrer le bootstrapping
bootstrap/apps/cert-manager-app.yaml	apiVersion: argoproj.io/v1alpha1 kind: Application metadata: name: cert-manager namespace: argocd annotations: argocd.argoproj.io/sync-wave: '-2' # installé en premier spec: source: repoURL: https://charts.jetstack.io chart: cert-manager targetRevision: v1.14.0 helm: parameters: - name: installCRDs value: 'true' destination: server: https://kubernetes.default.svc namespace: cert-manager syncPolicy: automated: selfHeal: true	■ Oui — ArgoCD installe cert-manager lui-même via Helm. Sync Wave -2 garantit qu'il est prêt avant tout le reste.
cluster-issuers/letsencrypt-prod.yaml	apiVersion: cert-manager.io/v1 kind: ClusterIssuer metadata: name: letsencrypt-prod annotations: argocd.argoproj.io/sync-wave: '-1' # après cert-manager, avant les apps spec: acme: server: https://acme-v02.api.letsencrypt.org/directory email: admin@mon-domaine.com privateKeySecretRef: name: letsencrypt-prod-key solvers: - http01: ingress: class: nginx	■ Oui — ArgoCD déploie le ClusterIssuer. Si quelqu'un le supprime accidentellement, ArgoCD le recrée automatiquement (self-heal).
cluster-issuers/letsencrypt-dns.yaml	apiVersion: cert-manager.io/v1 kind: ClusterIssuer metadata: name: letsencrypt-dns annotations: argocd.argoproj.io/sync-wave: '-1' spec: acme: server: https://acme-v02.api.letsencrypt.org/directory email: admin@mon-domaine.com privateKeySecretRef: name: letsencrypt-dns-key solvers: - dns01: azureDNS: subscriptionID: '<sub-id>' resourceGroupName: rg-prod hostedZoneName: mon-domaine.com environment: AzurePublicCloud clientID: '<sp-client-id>' clientSecretSecretRef: name: azure-dns-config key: client-secret tenantID: '<tenant-id>'	■ Oui — Variante DNS-01 pour clusters privés ou certificats wildcard *.mon-domaine.com. Géré par ArgoCD comme tout autre manifest.
apps/mon-app/ingress.yaml (avec annotation auto-cert)	apiVersion: networking.k8s.io/v1 kind: Ingress metadata: name: mon-app-ingress namespace: mon-app annotations: # ArgoCD déploie cet Ingress depuis Git # cert-manager détecte l'annotation et émet le certificat cert-manager.io/cluster-issuer: 'letsencrypt-prod' nginx.ingress.kubernetes.io/ssl-redirect: 'true' annotations: argocd.argoproj.io/sync-wave: '3' # après cert-manager et ClusterIssuer spec: ingressClassName: nginx tls: - hosts: - mon-app.mon-domaine.com secretName: mon-app-tls-secret # créé automatiquement par cert-manager rules: - host: mon-app.mon-domaine.com http: paths: - path: / pathType: Prefix backend: service: name: mon-app-svc port: number: 80	■ Oui — ArgoCD déploie l'Ingress. L'annotation cert-manager.io/cluster-issuer déclenche l'émission du certificat par cert-manager. Le Secret TLS est créé automatiquement.

Dossier / Fichier	Contenu	Géré par ArgoCD ?
apps/mon-app/certificate.yaml (certificat explicite)	# Alternative : déclarer explicitement le Certificate # (utile pour contrôler durée, SANS, etc.) apiVersion: cert-manager.io/v1 kind: Certificate metadata: name: mon-app-tls namespace: mon-app annotations: argocd.argoproj.io/sync-wave: '2' # avant l'Ingress spec: secretName: mon-app-tls-secret duration: 2160h # 90 jours renewBefore: 360h # renouveler 15j avant dnsNames: - mon-app.mon-domaine.com - api.mon-domaine.com issuerRef: name: letsencrypt-prod kind: ClusterIssuer	■ Oui — ArgoCD déploie l'objet Certificate (CRD cert-manager). cert-manager le détecte et fait l'émission. Si supprimé manuellement : ArgoCD self-heal le recrée, cert-manager réemet.

Ordre de Déploiement avec Sync Waves — TLS de bout en bout

Wave	Ressource déployée par ArgoCD	Effet côté cert-manager / cluster
Wave -2	Application ArgoCD : cert-manager (chart Helm v1.14.0) → crée les pods cert-manager, les CRDs Certificate, ClusterIssuer...	cert-manager devient opérationnel dans le cluster. Sans cette étape, les objets Certificate et ClusterIssuer seraient rejetés (CRDs absents).
Wave -1	ClusterIssuer letsencrypt-prod ClusterIssuer letsencrypt-dns Secret azure-dns-config (credentials Azure DNS pour DNS-01)	cert-manager enregistre les issuers. Prêt à traiter toute demande de certificat des waves suivantes.
Wave 0	Namespace mon-app ServiceAccount mon-app-sa NetworkPolicy	Infrastructure de base du namespace. Aucun impact TLS à ce stade.
Wave 1	ConfigMap app-config ExternalSecret app-secrets (sync depuis Azure Key Vault)	Configuration et secrets applicatifs disponibles.
Wave 2	Certificate mon-app-tls (objet CRD cert-manager) → dnsNames: [mon-app.mon-domaine.com]	cert-manager détecte le Certificate et démarre le challenge ACME. Le Secret TLS mon-app-tls-secret est créé une fois le certificat émis (quelques secondes à 2 min).
Wave 3	Deployment mon-app (3 replicas) Service mon-app-svc	L'application démarre. Le Secret TLS est déjà présent dans le cluster.
Wave 4	Ingress mon-app-ingress (référence secretName: mon-app-tls-secret)	L'Ingress NGINX monte le certificat TLS déjà émis. Le trafic HTTPS est immédiatement opérationnel sans attente de challenge.

Self-Heal ArgoCD — Protection des Objets cert-manager

Scénario	Sans ArgoCD self-heal	Avec ArgoCD self-heal (syncPolicy.automated.selfHeal: true)
Suppression accidentelle du ClusterIssuer	Le ClusterIssuer disparaît. Les futurs renouvellements et nouvelles émissions échouent silencieusement. Découverte lors d'une expiration de certificat en production.	# ArgoCD détecte la dérive en ~3 min # et recrée automatiquement le ClusterIssuer # Pour voir les événements de self-heal : argocd app get mon-cluster-issuers # → SYNC STATUS: Synced (auto-corrected) # Historique argocd app history mon-cluster-issuers
Modification manuelle du ClusterIssuer (email changé)	La configuration dérive de Git. L'équipe ne sait plus quelle est la config réelle en production.	# ArgoCD détecte OutOfSync immédiatement # et revert la modification # Pour voir le diff avant correction : argocd app diff mon-cluster-issuers # → email: admin@mon-domaine.com (Git) # vs email: autre@domaine.com (cluster)
Expiration imminente du certificat	cert-manager renouvelle automatiquement. Mais si le ClusterIssuer a été altéré, le renouvellement échoue.	# Vérifier l'état des certificats kubectl get certificates -A # READY AGE EXPIRATION # True 89d 2024-04-15 # cert-manager renouvelle 15j avant # ArgoCD garantit que le ClusterIssuer # est toujours conforme à Git
Restauration après disaster (nouveau cluster)	Il faut recréer manuellement cert-manager, les ClusterIssuers, les Certificates, les Ingress... Processus long et risqué d'oubli.	# Avec ArgoCD GitOps : # 1. Créer le nouveau cluster AKS az aks create ... # 2. Installer ArgoCD helm install argocd argo/argo-cd ... # 3. Appliquer l'app racine kubectl apply -f bootstrap/root-app.yaml # → ArgoCD déploie cert-manager (wave -2), # les ClusterIssuers (wave -1), # les apps et leurs certificats # Tout est restauré automatiquement !

■ Règle d'or ArgoCD + cert-manager : cert-manager dans sa propre Application ArgoCD (Sync Wave -2) · ClusterIssuers dans une Application dédiée (Sync Wave -1) · Certificates/Ingress dans les Applications applicatives (Waves 2-4) · selfHeal: true sur toutes les Applications pour protéger les objets TLS · Tout dans Git = disaster recovery en un seul kubectl apply.